# Graph Neural Networks

## Midterm Examination

Name: _____ Student ID: _____

Time allowed: 120 minutes **Total: 100 points**

**Instructions.**

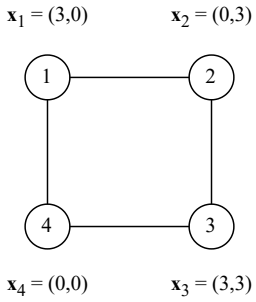
- Each response must include both the relevant formula(s) and a clear, **step-by-step** solution leading to the final answer.
- All answers must be written in **English**.

**Notation used throughout.** $\mathbf{A}$ is the adjacency matrix, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$, and the GCN propagation matrix is $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$. Node features are *row* vectors, so one GCN layer is $\mathbf{H}^{(l+1)} = \sigma(\hat{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$. $\mathcal{N}(v)$ is the set of neighbours of $v$ (excluding $v$), $d_v = |\mathcal{N}(v)|$. LeakyReLU has negative slope 0.2.

## Question 1: (Message Passing) 10 points

Consider the 4-cycle below. Each node has a 2-dimensional input feature $\mathbf{x}_v$ (shown next to it), and

$$\mathbf{X} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \\ 3 & 3 \\ 0 & 0 \end{pmatrix}, \qquad \mathbf{W} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}.$$




**(a)** **[5 pts]**

(i) Write down $\tilde{\mathbf{A}}$, $\tilde{\mathbf{D}}$ and the GCN propagation matrix $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$.

(ii) Compute the output of one GCN layer, $\mathbf{H}^{(1)} = \mathrm{ReLU}\big(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}\big)$. Give all four rows.

(iii) Now use the *basic neighbourhood-aggregation* layer instead, which keeps a separate weight for the node itself:

$$\mathbf{h}_v^{(1)} = \mathrm{ReLU}\Big(\big(\tfrac{1}{|\mathcal{N}(v)|}\textstyle\sum_{u\in\mathcal{N}(v)} \mathbf{x}_u\big)\mathbf{W} + \mathbf{x}_v\mathbf{B}\Big), \qquad \mathbf{B} = \mathbf{I}.$$

Compute $\mathbf{h}_1^{(1)}$ and $\mathbf{h}_4^{(1)}$.

**(b)** **[5 pts]**

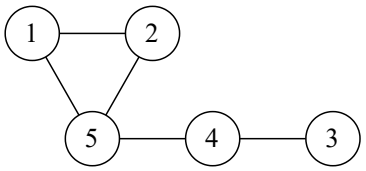
A node-level function $f(\mathbf{A}, \mathbf{X})$ is *permutation-equivariant* if $\mathbf{P}f(\mathbf{A}, \mathbf{X}) = f(\mathbf{P}\mathbf{A}\mathbf{P}^\top, \mathbf{P}\mathbf{X})$ for every permutation matrix $\mathbf{P}$. Prove that the GCN layer $f(\mathbf{A}, \mathbf{X}) = \mathrm{ReLU}(\hat{\mathbf{A}}\mathbf{X}\mathbf{W})$ is permutation-equivariant. Why does this property matter for graphs?

## Question 2: (Expressive Power) 10 points

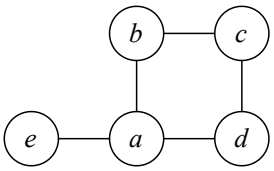
All nodes in the two graphs below have the *same* initial feature (colour). The 1-WL (colour refinement) update is

$$c^{(t)}(v) = \mathrm{HASH}\Big(c^{(t-1)}(v),\ \{\!\{\, c^{(t-1)}(u) : u \in \mathcal{N}(v) \,\}\!\}\Big),$$

where $\{\!\{\cdot\}\!\}$ is a multiset and HASH is injective. Two graphs are declared *non-isomorphic* as soon as their colour histograms differ.




**Graph $G$**




**Graph $H$**

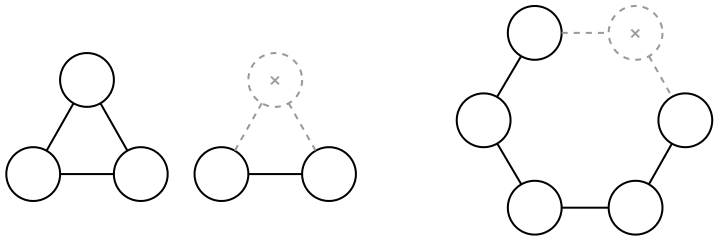
**(a)** **[5 pts]**

Run 1-WL on $G$ and $H$ in parallel (shared HASH table). For iterations $t = 1$ and $t = 2$, list each node's signature $\big(c^{(t-1)}(v), \{\!\{\cdots\}\!\}\big)$ and the resulting colour histogram of each graph.

**(b)** **[5 pts]**

Can a 2-layer mean-aggregation GNN distinguish these graphs when all initial node features are identical? Can a 2-layer GIN with sufficiently expressive, injective aggregation and graph-level readout distinguish them? Explain the difference. Justify in one or two lines.

## Question 3: (Expressive Power) 5 points



$G_1$ with one node deleted $G_2$ with one node deleted

**(a)** **[2 pts]**

**ESAN (Equivariant Subgraph Aggregation Networks)** does not run the GNN on the graph itself. It turns the graph into a *bag of subgraphs* with a selection policy, runs the GNN on every subgraph, and pools the results. Use the **node-deleted** policy: each subgraph is the graph with exactly one node (and its edges) removed.

For one node-deleted subgraph of $G_1$ and one of $G_2$, list the degree of every remaining node. Do the two subgraphs look different to 1-WL? Assuming an injective multiset aggregator and graph readout, explain why a node-deletion-based subgraph GNN can distinguish these graphs, even though a standard 1-WL-based GNN cannot distinguish their original structures.

**(b)** **[3 pts]**

Briefly describe each of the following ways to make GNNs more expressive than 1-WL. For each, give *one advantage* and *one disadvantage*.

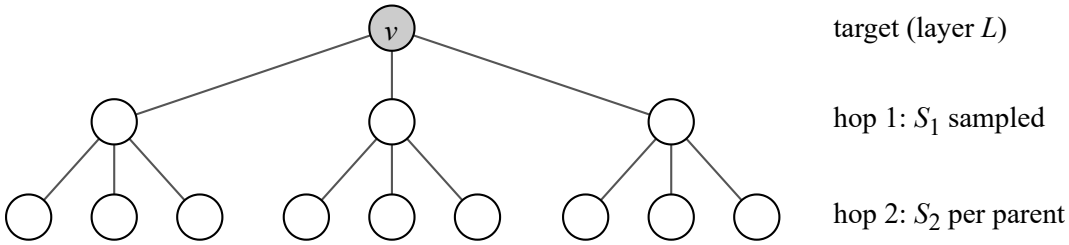
(i) Adding structural features, such as substructure counts (GSN), or random node features / node IDs.

(ii) Subgraph GNNs (e.g. ESAN): run the GNN on many slightly different subgraphs (for example, delete one node at a time) and combine the results.

(iii) Higher-order GNNs ($k$-GNN): pass messages between *groups* of $k$ nodes instead of single nodes.

## Question 4: (Scalability) 10 points

The figure shows how a node-wise sampler builds the computation tree for one target node $v$ (here $S_1 = S_2 = 3$).




**(a)** **[5 pts]**

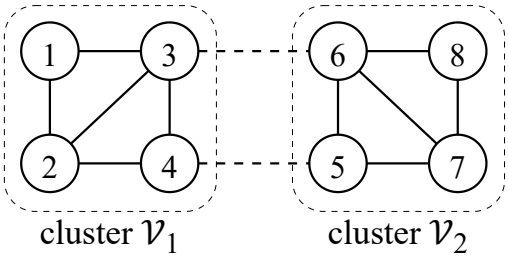
Assume that each expanded node has exactly 30 neighbors and that no nodes are revisited during neighborhood expansion, a mini-batch of $B = 32$ target nodes, and (worst case) no overlap between the neighbourhoods of different nodes. With *full* neighbourhood expansion, how many node embeddings must be computed for one target node and for the whole batch when the GNN has $L = 2$ layers?

**(b)** **[5 pts]**

Describe the two obstacles (memory and gradient updates) that make conventional full-batch training computationally challenging on graphs like `ogbn-papers100M` (111 M nodes, 1.6 B edges). Why are hub ("celebrity") nodes especially harmful?

## Question 5: (Scalability) 10 points

The graph below has $|\mathcal{V}| = 8$ nodes and $|\mathcal{E}| = 12$ edges. A METIS-style clustering [1] splits it into $\mathcal{V}_1 = \{1, 2, 3, 4\}$ and $\mathcal{V}_2 = \{5, 6, 7, 8\}$. Dashed edges cross the clusters.




**(a)** **[5 pts]**

Cluster-GCN writes $\mathbf{A} = \bar{\mathbf{A}} + \Delta$, where $\bar{\mathbf{A}}$ is block-diagonal (within-cluster edges) and $\Delta$ holds the between-cluster edges. How many edges are in $\Delta$, and what fraction of edges is lost when each mini-batch is one cluster ($q = 1$)? What happens with $q = 2$ clusters per batch?

**(b)** **[5 pts]**

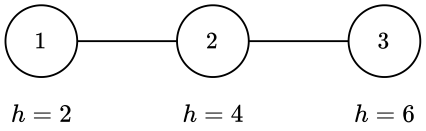
How does Cluster-GCN improve the scalability of GNN training through graph partitioning, and what are the potential limitations of restricting message passing to within clusters?

[1] METIS is a state-of-the-art graph partitioning algorithm that can generate partitions with minimal number of cross-partition edges, making it suitable for distributed message passing where the amount of network communication is proportional to the number of cross-partition edges.

## Question 6: (Deeper GNNs) 10 points

**(a)**

(i) Consider a three-node undirected graph with initial node features $h_1^{(0)} = 2$, $h_2^{(0)} = 4$, and $h_3^{(0)} = 6$.

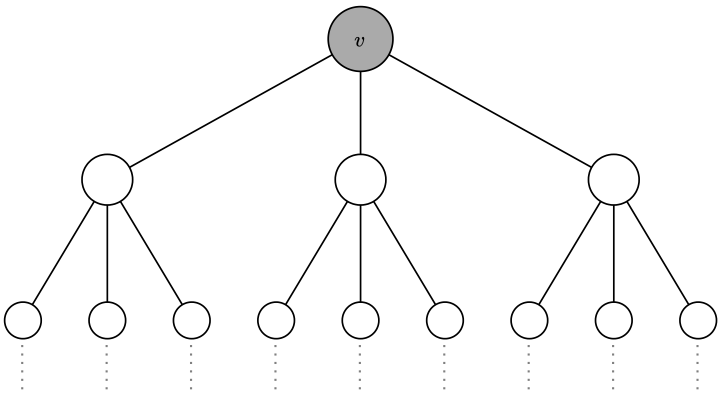



A GNN updates each node representation by averaging its own feature and those of its neighbors:

$$h_v^{(l+1)} = \frac{h_v^{(l)} + \sum_{u \in \mathcal{N}(v)} h_u^{(l)}}{|\mathcal{N}(v)| + 1}.$$

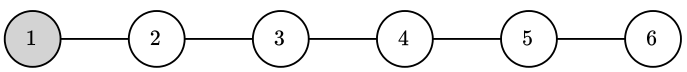
Calculate the node representations after two layers. Compare the differences between neighboring node representations before and after propagation. What does this suggest about over-smoothing?

(ii) Consider a tree graph in which each node has 3 children. A message-passing GNN aggregates information from neighboring nodes into a fixed-dimensional representation $h_v^{(l)} \in \mathbb{R}^4$.




For a 4-layer GNN, calculate the total number of nodes in the root's receptive field (including the root) and explain why increasing the number of layers can cause over-squashing.

(iii) On the path below, the label of node 1 depends on the feature of node 5. Which nodes can a 2-layer GNN at node 1 see? What is the smallest number of layers that lets node 1 see node 5?




**(b)** Explain why repeated neighbourhood averaging acts like a *low-pass filter*?

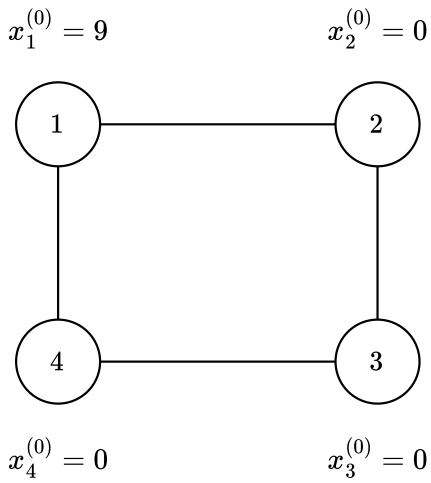
## Question 7: (Deeper GNNs) 10 points

**GCNII** introduces initial residual connections and identity mapping to mitigate over-smoothing and improve the optimization of deep GCNs, enabling effective training with dozens of layers. Each layer is

$$\mathbf{H}^{(l+1)} = \sigma\Big( \underbrace{\big((1-\alpha)\,\tilde{\mathbf{P}}\mathbf{H}^{(l)} + \alpha\,\mathbf{H}^{(0)}\big)}_{\text{initial residual}} \underbrace{\big((1-\beta)\,\mathbf{I} + \beta\,\mathbf{W}^{(l)}\big)}_{\text{identity mapping}} \Big),$$

where $\tilde{\mathbf{P}}$ is the propagation step (average of a node and its neighbours), $\mathbf{H}^{(0)}$ are the initial features, and $\alpha, \beta \in [0,1]$. A plain GCN layer is $\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{P}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$.

**(a)** **[5 pts]**




Consider the 4-cycle graph shown above with scalar features $\mathbf{H}^{(0)} = (9,0,0,0)$, so $\tilde{\mathbf{P}}\mathbf{H}^{(0)} = (3,3,0,3)$. The weight is the scalar $\mathbf{W}^{(0)} = 3$, $\sigma = \text{ReLU}$, $\alpha = \frac{1}{3}$ and $\beta = \frac{1}{2}$.

(i) **Initial residual.** Compute $(1-\alpha)\tilde{\mathbf{P}}\mathbf{H}^{(0)} + \alpha\mathbf{H}^{(0)}$.

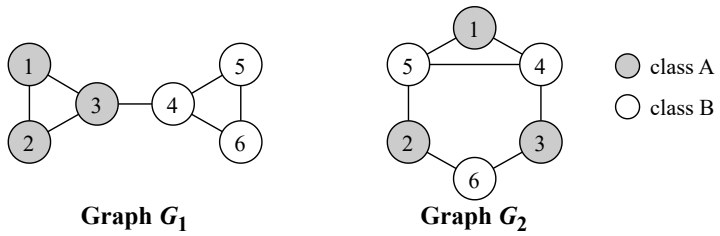
(ii) **Identity mapping.** Compute the factor $(1-\beta) + \beta\mathbf{W}^{(0)}$, and then the GCNII output $\mathbf{H}^{(1)}$.

**(b)** **[5 pts]**

Explain how the initial residual and identity mapping in GCNII help mitigate over-smoothing and facilitate training deeper GNNs. Why does repeatedly incorporating the initial features help preserve node-specific information?

## Question 8: (Heterophily) 10 points

The two graphs each contain six nodes:




**Graph $G_1$** **Graph $G_2$**

**(a).** **[5 pts]**

For **both** graphs, compute the following:

(i) Edge homophily $h_{edge} = \dfrac{|\{(u,v) \in \mathcal{E} : y_u = y_v\}|}{|\mathcal{E}|}$.

(ii) Node homophily $h_{node} = \dfrac{1}{|\mathcal{V}|} \sum_v \dfrac{|\{u \in \mathcal{N}(v) : y_u = y_v\}|}{d_v}$.

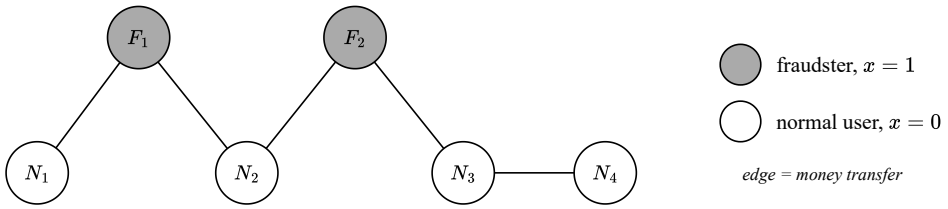
**(b)** **[5 pts]**

Four common homophily metrics are

$$h_{edge}, \qquad h_{node}, \qquad h_{class} = \frac{1}{C} \sum_{k=1}^{C} \max\Big(0,\ h_k - \frac{|\mathcal{V}_k|}{|\mathcal{V}|}\Big), \qquad h_{adj} = \frac{h_{edge} - \sum_k p_k^2}{1 - \sum_k p_k^2},$$

where $h_k$ is the fraction of edges leaving class-$k$ nodes that stay in class $k$, and $p_k = \sum_{v:y_v=k} d_v / (2|\mathcal{E}|)$. On the same graph these four metrics can give very different numbers. **Why do the metrics disagree?**

## Question 9: (Heterophily) 10 points




**(a)** **[5 pts]**

For this exercise, consider a simplified H2GCN-inspired aggregation using exact-distance one-hop and two-hop neighborhoods. (i) keeps the node's own feature *separate* from its neighbours (no self-loop mixing) and (ii) aggregates 1-hop and 2-hop neighbours in *separate* channels, then concatenates them. Using the mean as aggregator:

$$\mathbf{r}_v = \left[ x_v \parallel \text{mean}_{u \in \mathcal{N}_1(v)} x_u \parallel \text{mean}_{u \in \mathcal{N}_2(v)} x_u \right].$$

Compute $\mathbf{r}_v$ for all six nodes.

**(b)** **[5 pts]**

H2GCN is built on three designs: **(D1)** ego–neighbour separation, **(D2)** higher-order neighbourhoods, and **(D3)** combining intermediate representations. Explain why each one helps under heterophily.

## Question 10: (Attentive GNNs) 10 points

**(a)** **[5 pts]**

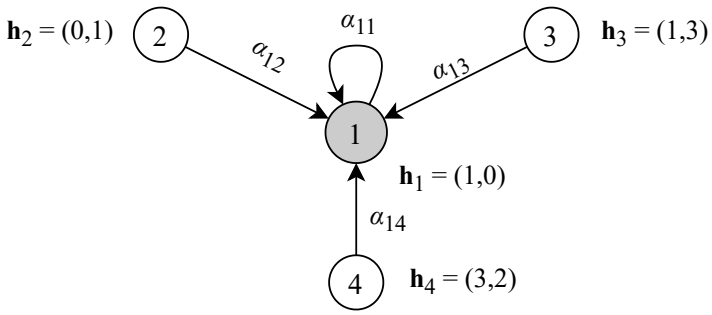
A GAT layer updates every node $i$ by attending to its neighbours and to itself (GAT adds a self-loop, so the attended set is $\tilde{\mathcal{N}}(i) = \mathcal{N}(i) \cup \{i\}$):

$$e_{ij} = \text{LeakyReLU}\left(\mathbf{a}^\top [\,\mathbf{W}\mathbf{h}_i \,\|\, \mathbf{W}\mathbf{h}_j\,]\right), \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k\in\tilde{\mathcal{N}}(i)} \exp(e_{ik})}, \qquad \mathbf{h}'_i = \sigma\Big(\sum_{j\in\tilde{\mathcal{N}}(i)} \alpha_{ij}\,\mathbf{W}\mathbf{h}_j\Big),$$

where $\|$ is concatenation, $\mathbf{W}$ is a shared weight matrix and $\mathbf{a}$ is the attention vector. In the graph below, node 1 (shaded) has neighbours 2, 3 and 4. Use

$$\mathbf{W} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \qquad \mathbf{a} = (1.4,\ 0,\ -0.7,\ -0.7)^\top, \qquad \text{LeakyReLU slope } 0.2, \qquad \sigma = \text{ReLU}\,.$$

**Use $e^{0.7} \approx 2$, $e^{0} = 1$ and $e^{-0.7} \approx \frac{1}{2}$.**




Compute one GAT layer for node 1.

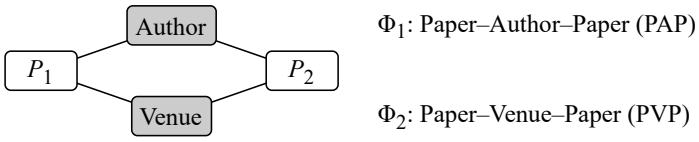
Show each step: the transformed features $\mathbf{W}\mathbf{h}_j$; the raw scores $\mathbf{a}^\top[\mathbf{W}\mathbf{h}_1 \| \mathbf{W}\mathbf{h}_j]$ and the attention scores $e_{1j}$ after LeakyReLU; the attention weights $\alpha_{1j}$; and the output $\mathbf{h}'_1$.

**(b)** **[5 pts]**

What is the static attention limitation of the original GAT? Explain how GATv2 modifies the attention mechanism to address this limitation, and why this modification improves the model's ability to capture query-dependent relationships between nodes.

## Question 11: (Attentive GNNs) 5 points

In an academic graph, two papers can be connected through two different *meta-paths*:




**(a)** **[2 pts]**

HAN's semantic-level attention gives the scores $w_{\Phi_1} = 1.4$ (PAP) and $w_{\Phi_2} = 0.7$ (PVP). For a particular paper, its meta-path-specific embeddings are $\mathbf{z}^{\Phi_1} = (3, 0, 6)$ and $\mathbf{z}^{\Phi_2} = (0, 3, 3)$. Using $e^{0.7} \approx 2$ and $e^{1.4} \approx 4$, compute the meta-path weights $\beta_{\Phi_p} = \exp(w_{\Phi_p}) / \sum_{p'} \exp(w_{\Phi_{p'}})$ and the final embedding $\mathbf{z} = \sum_p \beta_{\Phi_p} \mathbf{z}^{\Phi_p}$. Which relation does the model trust more?

**(b)** **[3 pts]**

HAN uses two levels of attention. What does *node-level* attention learn, and what does *semantic-level* attention learn? Why are both needed?

*— End of Exam —*